Tharun H 231901055

Exp No : 10 Troubleshooting Fabric Applications

Aim

To identify, analyze, and resolve common issues encountered in Hyperledger Fabric applications by troubleshooting network connectivity, chaincode deployment, channel configuration, certificate authentication, and Docker container errors.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Theory

Troubleshooting is an essential part of administering Hyperledger Fabric networks. Errors may occur due to incorrect configuration, network failures, certificate issues, chaincode deployment failures, or Docker container problems.
Common troubleshooting techniques include:

- Checking Docker container status
- Viewing peer and orderer logs
- Verifying channel membership
- Checking chaincode installation
- Validating MSP and certificates
- Restarting Fabric services
- Cleaning Docker resources

Proper troubleshooting minimizes downtime and ensures reliable blockchain application deployment.

Procedure

Step 1: Navigate to the Fabric Test Network

cd ~/fabric-dev/fabric-samples/test-network

Step 2: Start the Hyperledger Fabric Network
./network.sh up createChannel -ca

Expected Output
Creating channel 'mychannel'

Starting peer0.org1

Starting peer0.org2

Starting orderer.example.com

Fabric Network Started Successfully

Step 3: Verify Docker Containers
Check whether all required containers are running.

docker ps

Expected Output
peer0.org1.example.com

peer0.org2.example.com

orderer.example.com

ca_org1

ca_org2

Step 4: Check Peer Logs
Display the logs of Peer Organization 1.
docker logs peer0.org1.example.com

Expected Output
Peer Started Successfully

Listening on Port 7051

Joined Channel Successfully

Step 5: Check Orderer Logs
docker logs orderer.example.com

Expected Output
Orderer Started Successfully

Receiving Transactions

Creating New Blocks

Step 6: Verify Channel Membership
peer channel list

Expected Output
Channels peers has joined:

mychannel

Step 7: Verify Installed Chaincode
peer lifecycle chaincode queryinstalled

Expected Output
Package ID:

basic_1.0

Version:1.0

Step 8: Troubleshoot Chaincode Deployment
Redeploy the chaincode if deployment has failed.

```
./network.sh deployCC \
-ccl javascript \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript
```

Expected Output
Packaging Chaincode

Installing Chaincode

Approving Chaincode

Committing Chaincode

Chaincode deployed successfully

Step 9: Verify Chaincode Execution
Query the ledger to ensure that the chaincode is functioning correctly.
peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["GetAllAssets"]}'

Expected Output
Asset records displayed successfully.

Step 10: Check Docker Resource Usage
Monitor resource utilization.
docker stats

Expected Output
CONTAINER CPU % MEM USAGE

peer0.org1 3%

peer0.org2 2%

orderer 1%

Step 11: Restart the Fabric Network
Stop and restart the network.
./network.sh down

./network.sh up createChannel -ca

Expected Output
Network Restarted Successfully

Step 12: Clean Docker Resources (Optional)
Remove unused Docker resources.
docker system prune -f

Expected Output
Deleted Containers

Deleted Networks

Deleted Images

Total reclaimed space: xxx MB

Tharun H 231901055

Common Errors and Solutions
Error Message Possible Cause Solution

Error Message Possible Cause Solution
Docker daemon not running Docker service stopped sudo systemctl start docker
Peer connection failed Peer container stopped Restart Fabric network
Chaincode deployment
failed

Incorrect chaincode path Verify -ccp directory

Channel not found Channel not created

Run ./network.sh
createChannel
Access denied Invalid MSP or certificates Regenerate certificates
Chaincode not installed Installation failed Redeploy chaincode
TLS handshake failed Incorrect TLS certificate Verify TLS CA files
Port already in use

Another service using the
port

Stop conflicting process

Troubleshooting Workflow
Start Fabric Network
|
▼
Verify Docker Containers
|
▼
Check Peer Logs
|
▼
Check Orderer Logs
|
▼
Verify Channel
|

▼
Verify Chaincode

```
│
▼
Test Application
│
▼
Identify Error
│
▼
Apply Solution
│
▼
Restart Network
│
▼
Verify Successful Execution
```

Architecture Diagram

```
Client Application
│
▼
Hyperledger Fabric
│
┌────────────────┼────────────────┐
▼ ▼ ▼
Peer Org1 Peer Org2 Orderer
│ │ │
└────────────────┼────────────────┘
▼

Blockchain Ledger
│
Docker Containers
│
Logs & Monitoring
```

Sample Output

Fabric Network Started Successfully

Peer Verified Successfully

Orderer Verified Successfully

Channel Verified Successfully

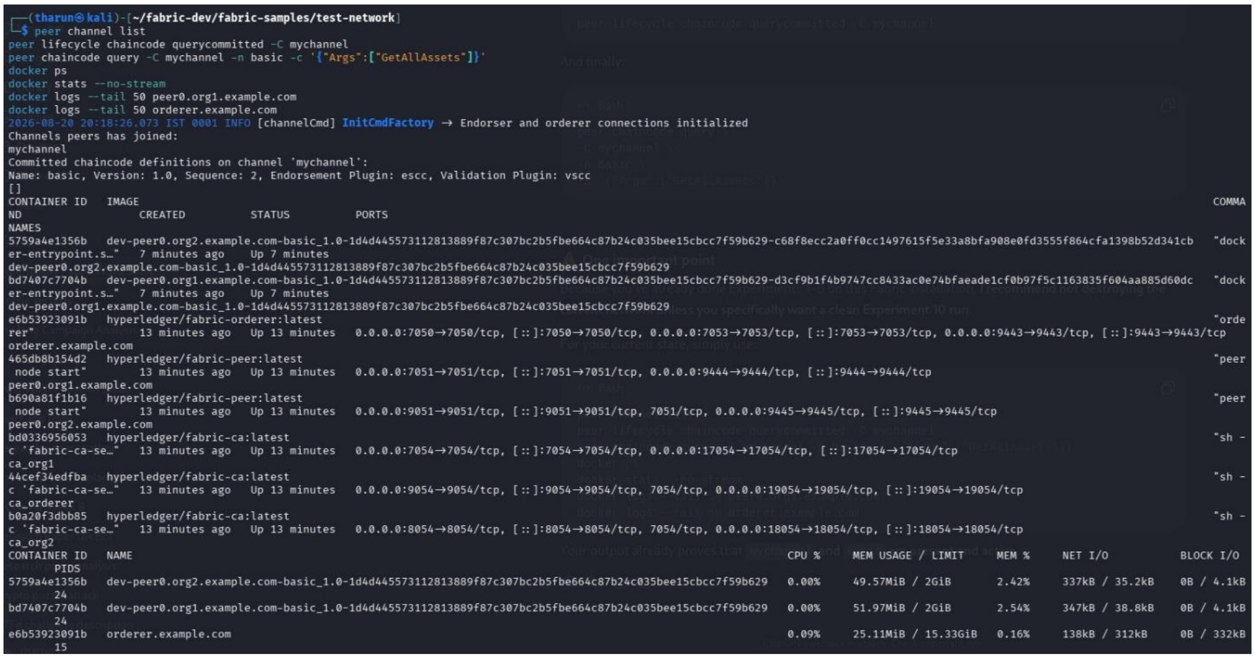
Tharun H 231901055

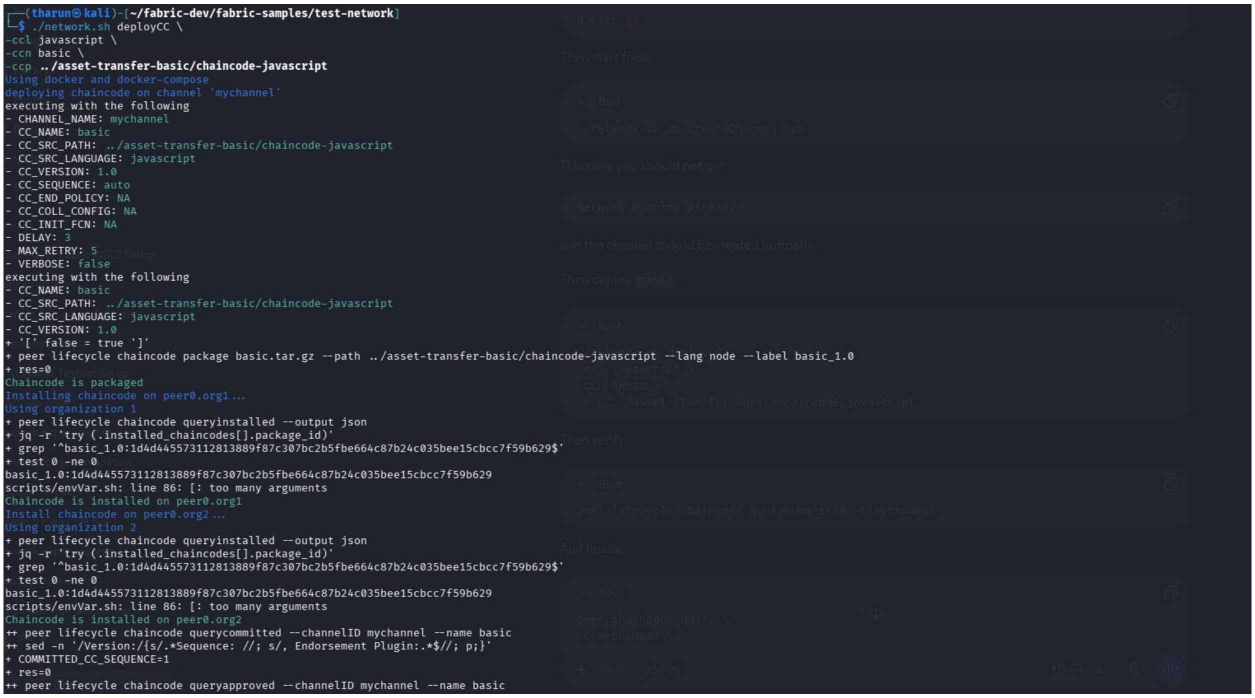
Chaincode Installed Successfully

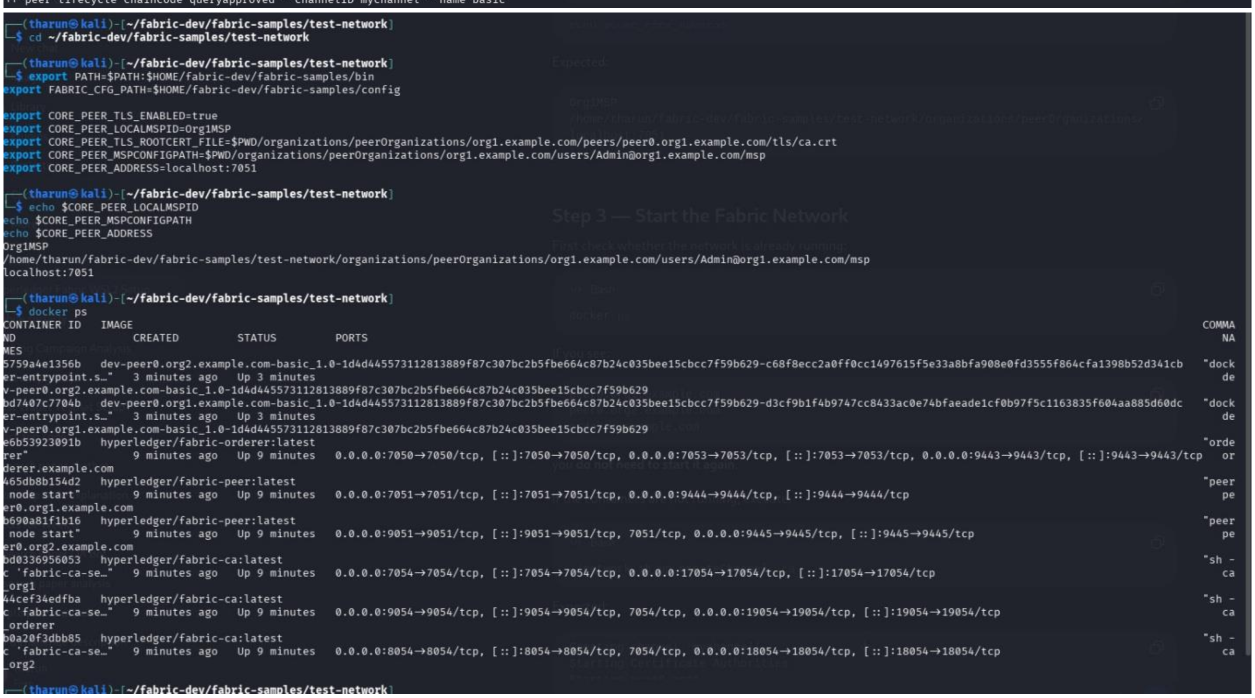
Application Executed Successfully

No Errors Detected

Fabric Network Running Normally







Result

The Hyperledger Fabric application was successfully analyzed and common operational issues were identified and resolved using Docker commands, Fabric CLI tools, and log analysis. The experiment demonstrated effective troubleshooting techniques for network connectivity, chaincode deployment, channel configuration, certificate management, and resource monitoring.